Zero-shot 2D detection under domain shift, and an LLM agent that tunes open-vocabulary prompts

CMPE249 Homework 1, Track A (experimental) · Maxim Dokukin · San José State University · October 2026

Code, configurations and command log: github.com/maxdokukin/class-cmpe-249-hw-1

Abstract. I ran six COCO-pretrained detectors from five families (YOLO11, YOLO26, Faster R-CNN, RT-DETR, OWLv2 and Grounding DINO; eight configurations in total) on COCO val2017 and, without retraining, on the 3,769-image KITTI validation split, all on a fanless Apple M4 laptop. On COCO the models span 0.47 to 0.53 mAP; on KITTI they collapse to 0.20 to 0.24, and the ranking changes. The drop is not uniform: car AP barely moves, while truck AP falls from about 0.44 to 0.13 and person to pedestrian from about 0.62 to 0.25. Input resolution shifts the KITTI ranking as much as architecture: YOLO11m at 1280 px matches Faster R-CNN. Error decomposition shows that missed objects cost the closed-set models the most AP, and that the open-vocabulary OWLv2 finds cyclists but draws boxes of the wrong extent. Finally, I let LLM agents (Claude Haiku 4.5, Haiku 5.5, Sonnet 5.5 and Opus 5.5) tune OWLv2's text prompts on a disjoint KITTI split. On the held-out test split, all 12 agent runs improved on the plain class names, by 0.2 to 1.3 mAP points, and 11 of the 12 gains are significant in a paired bootstrap. The runs cost $1.86 in total. The largest model was not the best optimizer, and no prompt fixed the cyclist class.

1 Question and setup

The homework asks whether I can design, reproduce and critically analyse a detection experiment. My question has three parts. How much accuracy do COCO-pretrained detectors lose on a driving dataset without retraining? Where does the loss come from: class, object size, occlusion, label taxonomy or input resolution? And can an LLM agent recover the part of the loss that comes from vocabulary, without changing any weights?

I built a small evaluation harness of my own (`src/hw1/` in the repository) instead of running the course's `ngdet` harness [1] directly. Four of its defaults would bias the numbers that I wanted: a 0.3 score threshold before scoring, dropped KITTI DontCare regions, bicycle boxes scored as cyclists, and a 120-image evaluation. §5.3 measures the effect of the first. I reused its structure, and its hand-written prompt synonyms serve as a baseline.

1.1 Models

Table 1 lists the configurations. I chose them to cover five families and the distinctions that the rubric asks about: anchor-based two-stage (Faster R-CNN v2 [2], [3], [4]), anchor-free one-stage CNNs descended from YOLO [5] (YOLO11m [6] with NMS, and YOLO26m [7], which ships a one-to-many head used with NMS by default and an optional one-to-one head that needs no NMS; I ran both), a real-time DETR [8] with set prediction and no NMS (RT-DETR R50 [9]), and two open-vocabulary detectors: OWLv2 [10], which scores image patches against text embeddings, and Grounding DINO-T [11], which fuses text into a DETR decoder. All weights are public COCO-trained checkpoints from Ultralytics, torchvision [12] and Hugging Face transformers [13], [14], [15]. I used the tiny Grounding DINO, pre-trained on Objects365, GoldG and Cap4M, which is the configuration that its paper evaluates zero-shot on COCO (48.4 AP) [11].

1.2 Datasets, splits and label mapping

COCO val2017 [16] (5,000 images, 80 classes) is the source domain. OWLv2 and Grounding DINO take 0.8 to 2.6 s per image on my hardware, so I ran them on a seeded 1,000-image subset and also re-scored the closed-set models on those same images for a fair comparison (Table 2). **KITTI 2D object** [17] is the target domain. I use the standard 3DOP split [18] (3,712 train / 3,769 val images, sequences disjoint) as published by OpenPCDet [19]: the 3,769 val images are my test set, and a seeded 500-image subset of the train half is the development set that only the agent sees (§6). KITTI and COCO label objects differently, so I converted KITTI to COCO-format ground truth with explicit rules (`taxonomy.py`):

- **Car and Van** → car. A COCO model has no van class; as an ablation I made Van an ignore region instead (§5.3).
- **Truck** → truck, **Pedestrian** → person, **Tram** → train (open-vocabulary models are asked for "tram" directly).
- **DontCare** (far or unlabeled objects) becomes an ignore region (`iscrowd=1`) for every class, so a detection inside it is neither rewarded nor penalised. **Misc** is an ignore region for car and truck, and **Person_sitting** for pedestrian.
- **Cyclist** is one box around rider and bicycle, while a COCO model outputs a person box and a bicycle box. I pair each

Config	Family	Checkpoint	Input on KITTI (1242×375)	Score thr.	NMS IoU	Device
YOLO11m	1-stage CNN, anchor-free	`yolo11m.pt`	letterbox 640 → 640×192	0.001	0.7	MPS
YOLO11m @1280	same	`yolo11m.pt`	letterbox 1280 → 1280×384	0.001	0.7	MPS
YOLO26m (NMS-free)	1-stage CNN, one-to-one head	`yolo26m.pt`, `nms=False`	letterbox 640 → 640×192	0.001	none	MPS
YOLO26m (NMS)	same, default head	`yolo26m.pt`	letterbox 640 → 640×192	0.001	0.7	MPS
Faster R-CNN v2	2-stage, anchor-based	`..._V2_Weights.COCO_V1`	short side 800, cap 1333 → 1333×402	0.001	0.5	MPS
RT-DETR R50	DETR, query-based	`PekingU/rtdetr_r50vd`	squash to 640×640	0.0	none	MPS
OWLv2 B/16	open-vocab, ViT	`owlv2-base-patch16-ensemble`	pad to square, 960×960	0.001	none	MPS
Grounding DINO-T	open-vocab, DETR	`grounding-dino-tiny`	short side 800, cap 1333 → 1333×402	0.001	none	CPU

Table 1: Configurations. Every model keeps at most 100 detections per image. The low score threshold keeps the full precision-recall curve (§5.3). Open-vocabulary models are prompted with the KITTI class names (car, van, truck, pedestrian, cyclist, tram) on KITTI and with the 80 COCO names on COCO.

Config	COCO mAP	COCO-1k	KITTI mAP (95% CI)	AP50	APs	APm	APl	AR@100	ms (FPS)
YOLO11m	0.505	0.544	0.217 [0.208, 0.229]	0.440	0.146	0.208	0.288	0.365	20 (49)
YOLO11m @1280	–	–	0.237 [0.229, 0.250]	0.491	0.144	0.242	0.288	0.394	64 (16)
YOLO26m (NMS-free)	0.520	0.551	0.232 [0.224, 0.244]	0.481	0.169	0.226	0.295	0.439	20 (50)
YOLO26m (NMS)	0.521	–	0.223 [0.215, 0.234]	0.456	0.132	0.211	0.307	0.350	21 (49)
Faster R-CNN v2	0.469	0.494	0.239 [0.231, 0.251]	0.493	0.154	0.249	0.287	0.457	337 (3.0)
RT-DETR R50	0.529	0.559	**0.241** [0.232, 0.253]	0.480	0.180	0.238	0.307	0.454	187 (5.3)
OWLv2	–	0.498	0.195 [0.186, 0.207]	0.412	0.130	0.207	0.275	0.379	866 (1.2)
Grounding DINO-T	–	0.509	0.232 [0.223, 0.245]	0.421	**0.244**	0.243	0.269	**0.477**	1395 (0.7)^{CPU}

Table 2: Main results. COCO mAP is over 80 classes on all 5,000 images; COCO-1k is the same seeded 1,000-image subset for every model. KITTI metrics are over the 5 mapped classes on the 3,769 test images. Latency is the sustained median on KITTI on the M4 GPU, except Grounding DINO (M4 CPU). Peak MPS memory was 0.15 to 1.25 GB on KITTI (Table 4, appendix).

predicted bicycle with the predicted person that overlaps it from above and output their union box with the geometric mean score. Cyclist boxes are also ignore regions for pedestrian, so a rider found as "person" is not a false positive.

The KITTI test split then has 16,002 car, 606 truck, 2,280 pedestrian, 893 cyclist and 287 tram boxes.

1.3 Metrics, hardware and reproducibility

I score every run with pycocotools [20]: mAP@[.5:.95], AP50, AP75, AP for small/medium/large boxes, AR@1/10/100 and per-class AP. On KITTI I add the KITTI difficulty views (easy/moderate/hard via the benchmark's height, occlusion and truncation limits, applied as ignore regions) and KITTI-style AP at IoU 0.7 for cars and 0.5 for pedestrians and cyclists; these use COCO's 101-point interpolation and are not comparable with the KITTI leaderboard. Confidence intervals are 95% bootstrap intervals over test images [21] (500 resamples on KITTI, 200 on COCO); a test reproduces pycocotools' AP exactly from the same per-image match tables. I measured latency at batch size 1 including pre- and post-processing, with device synchronisation, and report the median of the first 200 timed images (**burst**) and of the whole run (**sustained**), because the fanless laptop throttles. Memory is the peak of `torch.mps.driver_allocated_memory()`, a unified-memory figure that is not comparable with CUDA memory.

All runs used a MacBook Air M4 (24 GB unified memory, macOS 15.7.7) with Python 3.12.15, PyTorch 2.14.1, torchvision 0.29.1, Ultralytics 8.4.190 and transformers 5.21.0, pinned by `uv.lock`. Before the main runs, each model ran on 20 COCO images on both the CPU and the MPS GPU; five of six gave identical mAP to four decimals. Grounding DINO's check was interrupted by hand, my pipeline then defaulted it to the CPU, and its latency is therefore a CPU number (marked in all tables). Each experiment is one command (Table 5 in the appendix); `run.sh` runs the whole pipeline and every executed command is appended to `runs/COMMANDS.md` with host, device and wall time. Each run also writes a metadata file with the model configuration, library versions, a hash of the source code, latency and memory.

2 Results

Table 2 and Figure 1 summarise the main runs. On COCO, the closed-set models lead: on the shared 1,000 images RT-DETR (0.559), YOLO26m (0.551) and YOLO11m (0.544) beat Grounding DINO (0.509) and OWLv2 (0.498), close to the published full-set results that I reproduce within 1 point (RT-DETR 0.529 against 53.1 AP [9], YOLO11m 0.505 against 51.5 [6], Faster R-CNN v2 0.469 against 46.7 [12]). On KITTI every model falls by more than half, and the spread between models shrinks from 6.0 to 4.6 points. The one-stage YOLOs run at 20 ms per image, RT-DETR at 187 ms, Faster R-CNN at 337 ms and OWLv2 at 866 ms on the M4 GPU.

3 Analysis 1: cross-dataset generalization

The loss is concentrated in two classes (Figure 2). Car AP barely changes (0.50 to 0.51 on COCO, 0.48 to 0.51 on KITTI for the closed-set models; RT-DETR even gains slightly). Truck AP falls from 0.42–0.46 to 0.11–0.14, and person AP falls from 0.60–0.64 to pedestrian AP of 0.24–0.26. The truck drop is partly taxonomy: COCO's "truck" includes pickups and vans, while KITTI's Truck means large lorries and its vans are scored as cars, so a COCO "truck" on a van counts as a class confusion. For the truck class alone, fixing class confusions would add 0.29–0.42 AP50 for the closed-set models, more than their whole truck AP50 (0.17–0.21): most of their truck detections sit on ground truth of another class, most likely the car class, which in KITTI includes vans. The pedestrian drop is a scale and occlusion effect: most KITTI pedestrians are far away, and over all classes, recall for objects at KITTI occlusion level 2 ("largely occluded") is 0.65–0.76, against 0.91–0.96 for fully visible ones (`runs/analysis/summary.md`).

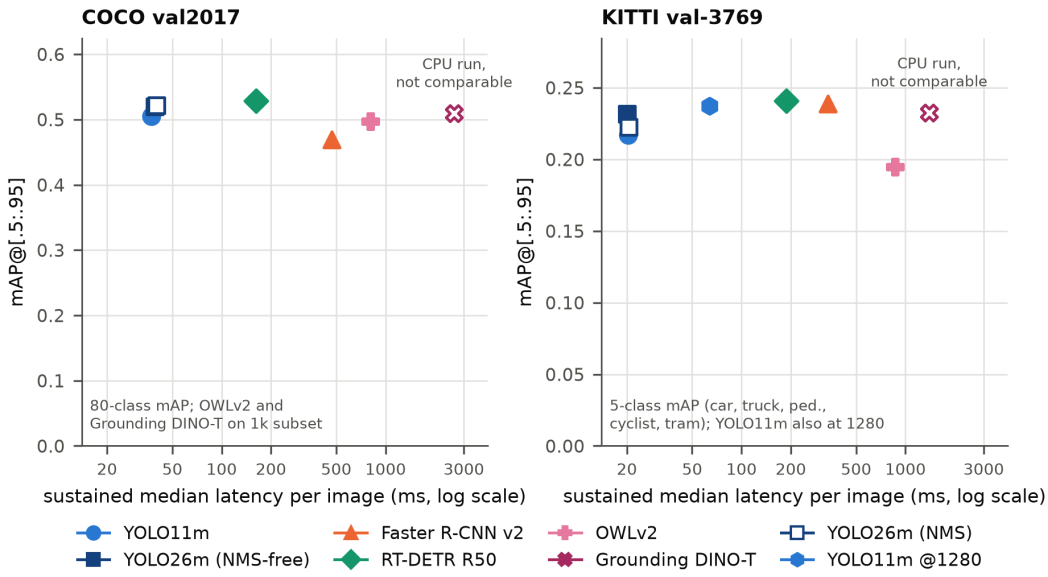
All five KITTI classes are shown in Figure 3. Car is the easiest class for every model (0.39–0.51), tram stays near 0.20–0.24, and cyclist is the hardest (0.02 for OWLv2, 0.06–0.10 for the others).

Small objects are found, but scored and localised badly. AP for small boxes halves from 0.31–0.38 on COCO to 0.13–0.18 on KITTI, yet recall@100 for objects under 25 px tall is 0.87–0.93, higher than for larger ones. The small-object problem on KITTI is therefore precision and box accuracy, not detection. Grounding DINO is the exception (APs 0.244), which higher resolution alone does not explain: YOLO11m's APs does not improve at 1280 px (0.146 → 0.144).

Difficulty views agree. KITTI-style car AP at IoU 0.7 drops from 0.90–0.93 (easy) to 0.61–0.64 (hard) for the closed-set models. OWLv2 trails at every level (0.73 → 0.44), and both open-vocabulary models are weak on pedestrians (moderate pedestrian AP 0.47–0.48 against 0.75–0.80 for the closed-set models) while Grounding DINO matches them on cars.

4 Analysis 2: architecture and resolution

Under domain shift, input resolution matters as much as architecture, at least for YOLO11m. On KITTI, Faster R-CNN v2, a two-stage design (2015, here with FPN and 2021 training recipes), scores 0.239, ahead of YOLO11m (0.217) and level with RT-DETR (0.241). This matches the course tutorial's observation [1], but resolution explains much

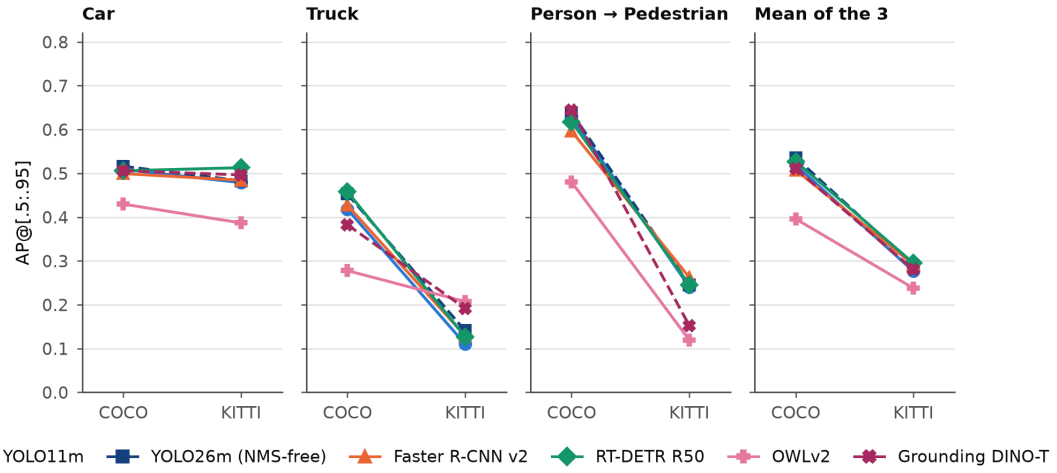


Grounding DINO-T was measured on the M4 CPU (an interrupted parity check), so its latency is not comparable to the MPS runs. OWLv2 and Grounding DINO-T COCO numbers are on a seeded 1000-image val2017 subset (the others use all 5000). Latency is batch 1 including pre/post-processing; the x range is shared. Hollow markers: YOLO26m with the NMS head (Ultralytics default; the solid YOLO26m run used the opt-in NMS-free head), and the CPU-measured Grounding DINO-T.

Figure 1: Speed against accuracy on the M4 (sustained median latency, log scale). Grounding DINO's latency is a CPU measurement and is not comparable. OWLv2 and Grounding DINO COCO points are on the 1,000-image subset.

of it: Faster R-CNN's resize rule upsamples KITTI frames to 1333×402, while YOLO's letterbox at 640 shrinks them to 640×192. Running the same YOLO11m at 1280 (1280×384) raises it to 0.237, within the confidence interval of Faster R-CNN, at 64 ms instead of 337 ms per image. The gain comes from medium objects (APm 0.208 → 0.242) and cyclists (0.059 → 0.108), not from small ones (APs 0.146 → 0.144). I tested resolution on one model only. RT-DETR reaches the best KITTI mAP even though its resize rule squashes the wide frame to 640×640, which changes aspect ratios by a factor of 3.3.

The NMS-free head may help under shift. YOLO26m with its optional one-to-one head (no NMS) and with its default NMS head are equal on COCO (0.520 vs 0.521; Ultralytics reports 52.5 and 53.1 [7]) but differ on KITTI (0.232 [0.224, 0.244] vs 0.223 [0.215, 0.234]; the intervals overlap, and each head ran once). The error decomposition (Figure 4) shows where: with NMS, missed objects cost 0.272 AP50; without it, 0.180, and recall@100 is 0.830 against 0.877. For cars alone the missed-object cost is 0.107 against 0.070. This is consistent with NMS at IoU 0.7, tuned on COCO, removing true detections of overlapping cars in KITTI's dense parking rows, which the one-to-one head does not suppress. The NMS-free head produces more duplicates (Dupe ΔAP50 0.005 vs 0.001), a small cost.



Same AP scale in all panels. 'Mean of the 3' is mAP over car, truck and person/pedestrian only (the classes both datasets share), so it differs from the headline 80-class / 5-class mAP. Dashed = second model of a family. OWLv2 and Grounding DINO-T COCO numbers are on a seeded 1000-image val2017 subset (the others use all 5000).

Figure 2: Per-class AP on COCO and on KITTI for the classes with a direct counterpart (person → pedestrian).

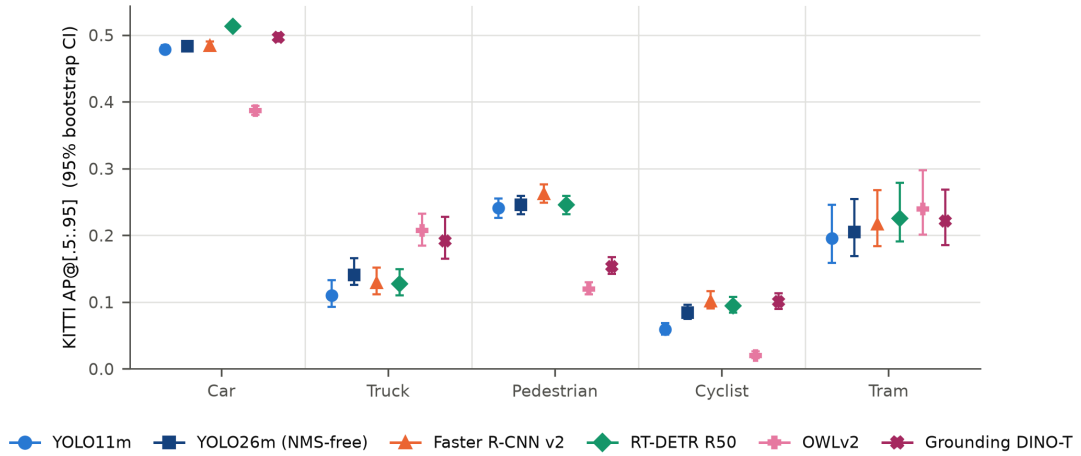


Figure 3: KITTI per-class AP@[.5:.95] with 95% bootstrap intervals. Full per-class tables for both datasets, including AR@1/10/100, are in `runs/tables/` (T1, T2) and `runs/metrics/`.

5 Analysis 3: speed–accuracy and failure modes

5.1 Speed and memory

YOLO11m and YOLO26m are 9–17× faster than RT-DETR and Faster R-CNN on the M4 for 0.9 to 2.4 points less KITTI mAP, and YOLO11m at 1280 px gives the best trade-off for KITTI (0.237 at 64 ms). Throttling is visible for the heavier models: RT-DETR's median rose from 116 ms (burst) to 187 ms (sustained) and OWLv2's from 635 to 866 ms, while the YOLOs stayed at 20 ms. Peak MPS memory follows input size rather than parameter count: YOLO11m used 0.16 GB on letterboxed KITTI frames and 1.16 GB on 640×640 COCO images, while RT-DETR and OWLv2, which resize to fixed squares, used the same memory on both.

5.2 Error decomposition

I adapted TIDE [22] to KITTI's ignore regions (`failure.py`). For the closed-set models, missed objects cost the most (0.15–0.27 AP50), then class confusion (0.07–0.12) and localisation (0.07–0.09). OWLv2's largest error is localisation (0.172). Grounding DINO's largest error is class confusion (0.125), which fits a label space defined by text. For the cyclist class the picture splits by model type (Figure 4, right): the closed-set models mostly miss cyclists even with my pairing rule (Missed 0.21–0.42), while OWLv2's cyclist loss is almost all localisation (Loc 0.341): it finds the cyclists but draws a box around only the rider or only the bicycle. This explains the result of §6 that no prompt fixed the cyclist class.

5.3 Effect of evaluation choices

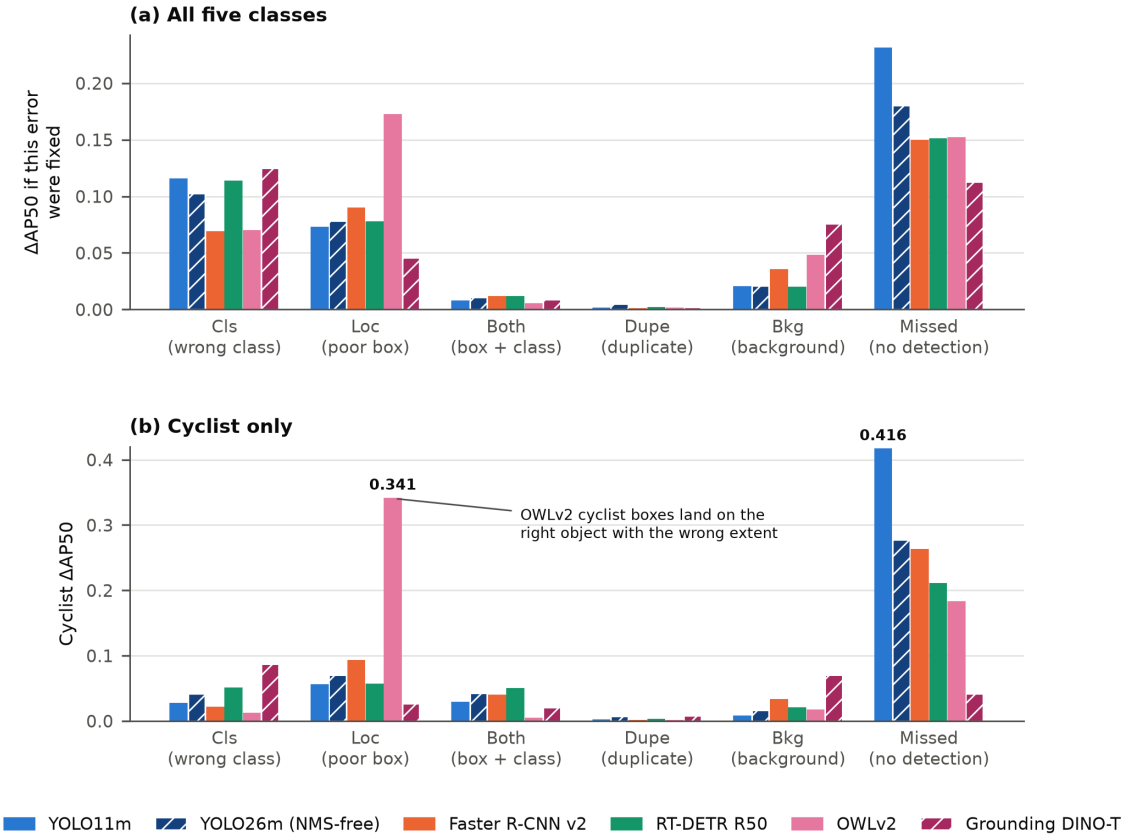
Three choices change the numbers by more than the gaps between models (full table: `runs/tables/T3_ablations.md`). A 0.3 score threshold before scoring, the default in the course harness, costs 1.1 to 3.5 points on KITTI and 1.6 to 8.2 points on COCO (YOLO26m: 0.520 → 0.438), because it truncates the precision-recall curve; models with low-scoring outputs lose most. Mapping only bicycle boxes to Cyclist gives cyclist AP of 0.000–0.001, the same as dropping the class, while my pairing rule gives 0.06–0.11 and adds 1.2–2.1 points of mAP. Treating Van as an ignore region instead of a car adds 0.4–0.6 points to every model and changes no ranking.

6 Analysis 4: closed-set vs open-vocabulary, and an LLM agent that tunes prompts

With plain class names as prompts, OWLv2 is the weakest model on KITTI (0.195) and Grounding DINO ties YOLO26m (0.232). Grounding DINO keeps a larger share of its COCO accuracy (46% against 40% for YOLO11m on the shared COCO subset), but both open-vocabulary models are poor at pedestrians (0.12–0.15 AP) and cyclists. An open-vocabulary detector's only adaptation knob is its vocabulary, so I asked whether an LLM agent could choose a better one.

Protocol. The agent proposes a prompt map: up to 5 prompts per class and 30 in total, each assigned to a KITTI class. Its only tool, `evaluate_prompt_map`, runs OWLv2 on the 500-image development split and returns per-class AP, true and false detections per prompt, and six image crops of the worst false positives and misses. It has 15 evaluations. The code, not the agent, keeps the map with the best development mAP, and that map is scored once on the 3,769-image test split, which the tool cannot reach. OWLv2's image features do not depend on the text, so I cached them once and each evaluation takes about a second; the cached path reproduces the full model exactly (raw names: 0.195 both ways). I ran four Claude models [23] three times each. Haiku 4.5 ran without the `effort` setting and without extended thinking; the others ran at `effort: medium`. I compare against the plain names, `ngdet`'s hand-written synonyms [1], and the agent's first proposal before any feedback (**one-shot**). Significance is a paired bootstrap of test mAP minus the plain-names mAP over the same 1,000 image resamples.

Results (Table 3, Figure 5). Every one of the 12 best maps beats the plain names on the test split, 11 with $P < 0.001$, by 0.17 to 1.34 points. `ngdet`'s synonyms are significantly worse than the plain names (−0.32), because "bicycle" detects only the bicycle and KITTI's cyclist box includes the rider. The feedback loop matters: one-shot proposals are no better than the plain names, except Sonnet's (+0.26 to +0.41 points,



TIDE-style oracle: each bar is AP50 gained (IoU 0.5, kitti_test, 3769 images) when only that error type is fixed, one oracle at a time; bigger = that error costs more. Loc = box overlaps a same-class object at IoU 0.1-0.5; Missed = ground truth never matched. Headline runs only (YOLO11m @1280 and YOLO26m (NMS) variants are in table T8). Closed-set models use the Cyclist merge; open-vocabulary models prompt Cyclist directly. AP50 here uses COCO-style 101-point interpolation and is not comparable to the KITTI leaderboard.

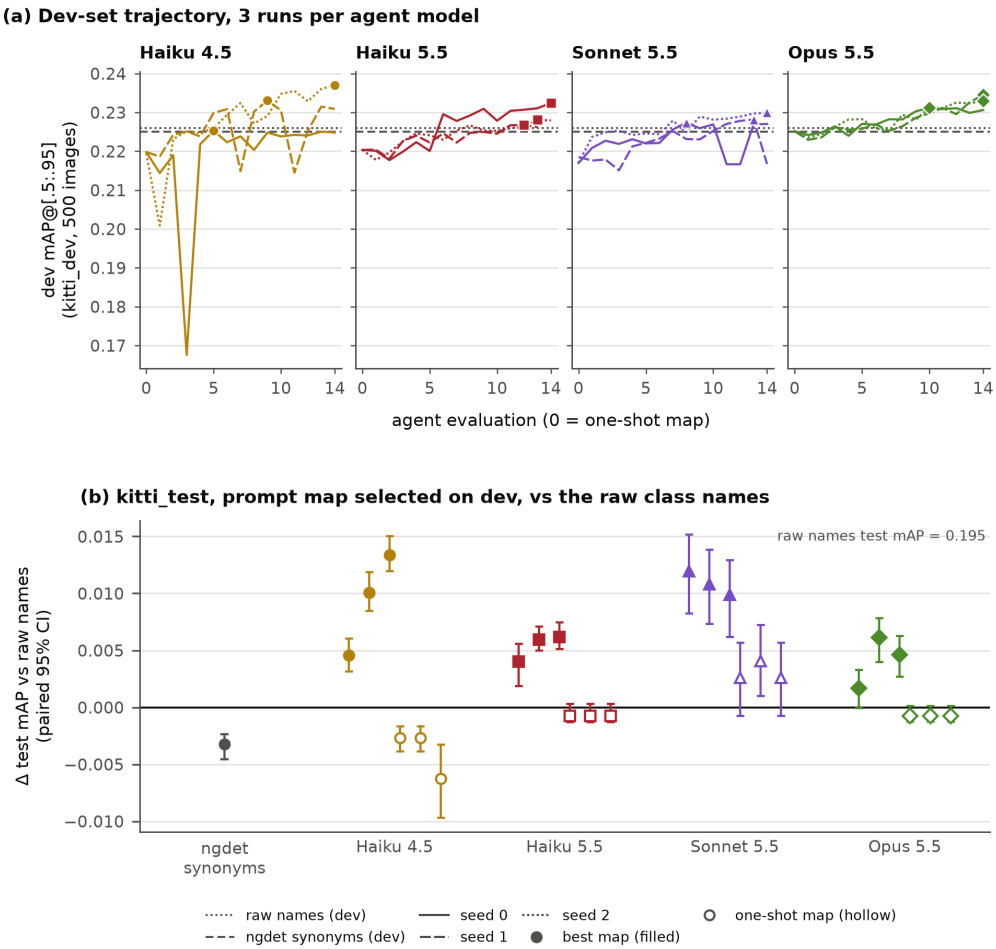
Figure 4: TIDE-style error decomposition on KITTI [22]: AP50 gained by fixing one error type with an oracle (Cls = right box, wrong class; Loc = right class, IoU 0.1–0.5; Both; Dupe = duplicate; Bkg = background; Missed = no detection). Right: the same for the cyclist class only.

significant in one of three runs), and the maps improve during the run. Test gains are not systematically smaller than development gains (for example Sonnet: +0.1 to +0.4 points on development, +1.0 to +1.2 on test), so I see no sign of overfitting the 500-image development split, whose estimates are noisy. The gains come from three sources: pose words for pedestrians ("standing person", "walking person"; test pedestrian AP 0.120 → up to 0.159), car subtypes ("minivan", "SUV", "dark car"; car AP up to 0.410), and, only in Sonnet's runs, tram synonyms ("light rail train", "streetcar"; tram AP 0.240 → 0.264–0.278). No model raised test cyclist AP above 0.030; each tried 13 to 21 different cyclist phrasings, and the best development cyclist AP was 0.021. This is consistent with the localisation diagnosis of §5.2: a prompt changes which boxes score high, not their extent.

Model size did not predict optimizer quality. Sonnet 5.5 had the largest and most consistent gains (about +1.1 points in all three runs). Opus 5.5, the most capable and most expensive model, had the smallest (+0.17 to +0.61), and Haiku 5.5 gave half of Sonnet's gain for 2 cents per run. The models also searched differently. Haiku 4.5 made large edits (mean change in development mAP of 0.005–0.010 per step). Replacing "truck" with "truck or van" cut development mAP by 5 points, and another run twice lost 1.6 points by replacing "truck" with "delivery truck" or "box truck"; each time it restored "truck" within one or two steps, and it reached the highest single test gain. The 5.x models made small, safe edits (0.001–0.003 per step, worst drop 0.011). With three runs per model, and gains of about one point, I do not rank the models beyond this; the paired intervals do not capture variation between runs.

Agent model	$/run	Output tok./run	Best dev mAP	Test Δ, best map (pts, runs 0/1/2)	Signif.	Test Δ, one-shot (pts)	Mean |dev step|
Haiku 4.5	0.07	4.7k	0.225–0.237	+0.46, +1.00, +1.34	3/3	−0.27, −0.27, −0.63	0.0051–0.0096
Haiku 5.5	0.01–0.02	12.3–15.3k	0.227–0.232	+0.40, +0.59, +0.62	3/3	−0.08 (all 3)	0.0014–0.0022
Sonnet 5.5	0.18–0.23	7.3–9.7k	0.227–0.230	**+1.19, +1.08, +0.99**	3/3	+0.26, +0.41, +0.26	0.0016–0.0027
Opus 5.5	0.30–0.35	7.4–8.9k	0.231–0.235	+0.17, +0.61, +0.46	2/3	−0.08 (all 3)	0.0011–0.0015
`ngdet` synonyms	–	–	0.225	−0.32 (CI −0.45 to −0.24)	worse	–	–

Table 3: Agent study on OWLv2 (test split: plain names score 0.195 mAP; dev: 0.226). "Signif." counts best maps whose paired 95% interval excludes zero (Opus run 0: P = 0.03, interval touches zero). Total API cost for all 12 runs: $1.86. The paired intervals cover image sampling only; the spread across runs of one model is larger.



(a) Larger marker = the evaluation the agent kept as its best map. (b) Within each model: best maps of seeds 0, 1, 2, then one-shot maps of seeds 0, 1, 2. Bars are paired bootstrap CIs (1000 image resamples of kitti_test) and reflect image sampling only, not run-to-run variance of the agent. 3 runs per model. Haiku 4.5 ran without the effort setting; the other three ran at effort medium. KITTI mAP is COCO-style mAP@[.5:.95] over the 5 mapped classes (car, truck, pedestrian, cyclist, tram); COCO mAP is over 80 classes.

Figure 5: (a) Development mAP after each of the 15 evaluations, per run, coloured by agent model, with the plain-names and `ngdet` baselines. (b) Test-split change in mAP against plain names, with paired 95% intervals.

7 Failure analysis

Figure 6 shows the failure modes behind the numbers. (1) **Occlusion**: parked cars in rows are partly hidden; recall falls from 0.91–0.96 to 0.65–0.76 at occlusion level 2. (2) **Taxonomy**: COCO models label many KITTI cars and vans "truck"; for the truck class, class confusion costs more AP50 than the class achieves, and KITTI's large trucks are rare (606 boxes). (3) **Box extent**: cyclists and riders produce boxes that cover part of the KITTI box; for OWLv2 this one error type costs 0.34 AP50 on cyclists. (4) **Scale**: small pedestrians are detected but with low scores and loose boxes. (5) **Threshold and NMS settings tuned on COCO**: YOLO26's NMS head misses more cars than its NMS-free head, consistent with NMS at IoU 0.7 removing true detections in dense rows of cars. The objects of at least 40 px that every closed-set model misses are of two kinds (`runs/analysis/shared.json`). One is a tram that KITTI labels as five adjacent boxes, one per carriage section, while a COCO model outputs one "train" box over the whole tram that matches none of them. The other is a group of partly occluded pedestrians, plus pedestrians cut off at the image edge (62% and 82% truncated).

Threats to validity. Grounding DINO's latency was measured on the CPU. OWLv2's ensemble checkpoint was fine-tuned on LVIS [10], whose images are COCO images, and its pseudo-labels came from an annotator trained on Objects365 and Visual Genome. Grounding DINO's pre-training data include GoldG [11], the grounding annotations collected by MDETR [24], which include Visual Genome, whose images come from COCO [25], so neither model's COCO score is strictly zero-shot; KITTI is. KITTI-style AP is computed with COCO's interpolation and is not leaderboard-comparable. The agent study uses 3 runs per model and one detector; Grounding DINO was too slow for 15 evaluations per run.

8 Conclusions

COCO-pretrained detectors lose more than half their accuracy on KITTI without retraining, and the loss sits in specific places: trucks (a taxonomy mismatch), distant and occluded pedestrians, and cyclists, whose KITTI box shape no COCO class matches. Car detection transfers almost intact. Under this shift, evaluation details and input resolution move the numbers as much as architecture: a 0.3 score threshold, a naive cyclist mapping or a 640-px letterbox each cost one to eight



Figure 6: Typical failures on KITTI test images. Green: ground truth (tagged MISS when no detection matched); orange: matched detections with score ≥ 0.3; red: false positives; magenta: objects that every closed-set model missed.

points, and the NMS-free YOLO26 head recovers objects that NMS suppresses in dense scenes. Open-vocabulary detectors do not fix domain shift by themselves, but their vocabulary is a cheap, tunable knob: LLM agents improved OWLv2 on held-out data in all 12 runs for under $2 in total, with a mid-size model doing best. The remaining error, box extent for cyclists, is beyond what prompts can change and would need fine-tuning on KITTI boxes.

Use of AI tools. Claude Code (Anthropic) wrote most of the harness, the analysis scripts and a first draft of this report under my direction; Claude models are also the subject of §6. Every number in this report comes from logged runs in `runs/` of the repository, and I checked the claims about other work against the cited papers and documentation.

References

[1] lkk688, *ngdet tutorial: zero-shot 2D detection across datasets (DeepDataMiningLearning)*. Accessed: Oct. 28, 2026. [Online]. Available: https://github.com/lkk688/DeepDataMiningLearning/blob/main/DeepDataMiningLearning/ngdet/TUTORIAL.md

[2] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[3] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[4] Y. Li, S. Xie, X. Chen, P. Dollár, K. He, and R. Girshick, "Benchmarking detection transfer learning with vision transformers," 2021. [Online]. Available: https://arxiv.org/abs/2111.11429

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[6] G. Jocher and J. Qiu, *Ultralytics YOLO11 (software and documentation)*. (2024). Accessed: Oct. 28, 2026. [Online]. Available: https://docs.ultralytics.com/models/yolo11/

[7] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," Jun. 2026. [Online]. Available: https://arxiv.org/abs/2606.03748

[8] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European Conference on Computer Vision (ECCV)*, 2020.

[9] Y. Zhao *et al.*, "DETRs beat YOLOs on real-time object detection," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

[10] M. Minderer, A. Gritsenko, and N. Houlsby, "Scaling open-vocabulary object detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[11] S. Liu *et al.*, "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *European Conference on Computer Vision (ECCV)*, 2024.

[12] PyTorch, "torchvision: fasterrcnn_resnet50_fpn_v2 (COCO_V1 weights, box mAP 46.7)." Accessed: Oct. 28, 2026. [Online]. Available: https://docs.pytorch.org/vision/main/models/generated/torchvision.models.detection.fasterrcnn_resnet50_fpn_v2.html

[13] Hugging Face, "Hugging Face Transformers documentation: RT-DETR." Accessed: Oct. 28, 2026. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/rt_detr

[14] Hugging Face, "Hugging Face Transformers documentation: Grounding DINO." Accessed: Oct. 28, 2026. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/grounding-dino

[15] Hugging Face, "Hugging Face Transformers documentation: OWLv2." Accessed: Oct. 28, 2026. [Online]. Available: https://huggingface.co/docs/transformers/model_doc/owlv2

[16] T.-Y. Lin *et al.*, "Microsoft COCO: Common objects in context," in *European Conference on Computer Vision (ECCV)*, 2014.

[17] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2012, pp. 3354–3361. doi: 10.1109/CVPR.2012.6248074.

[18] X. Chen *et al.*, "3D object proposals for accurate object class detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015, pp. 424–432.

[19] OpenMMLab, *OpenPCDet: data/kitti/ImageSets (train 3712, val 3769)*. Accessed: Oct. 28, 2026. [Online]. Available: https://github.com/open-mmlab/OpenPCDet/tree/master/data/kitti/ImageSets

[20] COCO Consortium, *cocodataset/cocoapi (pycocotools)*. Accessed: Oct. 28, 2026. [Online]. Available: https://github.com/cocodataset/cocoapi

[21] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[22] D. Bolya, S. Foley, J. Hays, and J. Hoffman, "TIDE: A general toolbox for identifying object detection errors," in *European Conference on Computer Vision (ECCV)*, 2020.

[23] Anthropic, "Claude API: Models overview." Accessed: Oct. 28, 2026. [Online]. Available: https://platform.claude.com/docs/en/models/overview

[24] A. Kamath, M. Singh, Y. LeCun, G. Synnaeve, I. Misra, and N. Carion, "MDETR: Modulated detection for end-to-end multi-modal understanding," 2021. [Online]. Available: https://arxiv.org/abs/2104.12763

[25] R. Krishna *et al.*, "Visual Genome: Connecting language and vision using crowdsourced dense image annotations," *International Journal of Computer Vision*, vol. 123, no. 1, pp. 32–73, 2017.

9 Appendix: commands and per-run details

Setup and full pipeline on the M4: `git clone https://github.com/maxdokukin/class-cmpe-249-hw-1; cd class-cmpe-249-hw-1; ./setup.sh; ./run.sh`. Stages are resumable: `parity e1 cache agent arms score analyze publish`. Agent runs: `AGENT_MODEL=claude-haiku-4-5 | claude-haiku-5-5 | claude-sonnet-5-5 | claude-opus-5-5 ./run.sh agent`, each `python -m hw1.agent --detector owlv2 --seed {0,1,2} --model …` with budget 15 and effort medium. Scoring: `python -m hw1.score_all` (bootstrap 500), `python -m hw1.arms --detector owlv2` (test arms and paired bootstrap), `python -m hw1.failure` (TIDE, recall bins, gallery), `python -m hw1.figures`.

Run	Split	Images	Device	Burst ms	Sustained ms	Peak MB	Wall min
Faster R-CNN v2	coco_val	5000	MPS	355.9	470.2	2250	40.8
Faster R-CNN v2	kitti_test	3769	MPS	267.5	337.3	1218	22.3
Grounding DINO-T	coco_val1k	1000	CPU	2526.7	2617.7	3427	43.2
Grounding DINO-T	kitti_test	3769	CPU	1379.3	1394.6	2438	91.4
OWLv2	coco_val1k	1000	MPS	650.3	808.8	1085	13.3
OWLv2	kitti_test	3769	MPS	634.5	865.9	1085	55.6
RT-DETR R50	coco_val	5000	MPS	116.7	161.7	1249	14.1
RT-DETR R50	kitti_test	3769	MPS	115.6	187.1	1249	12.4
YOLO11m	coco_val	5000	MPS	36.9	37.4	1156	3.9
YOLO11m	kitti_test	3769	MPS	20.1	20.3	162	1.8
YOLO11m (img1280)	kitti_test	3769	MPS	58.7	63.9	1146	4.6
YOLO26m (NMS-free)	coco_val	5000	MPS	38.0	39.0	1156	5.0
YOLO26m (NMS)	coco_val	5000	MPS	38.9	40.2	1156	3.9
YOLO26m (NMS-free)	kitti_test	3769	MPS	19.8	20.0	154	1.8
YOLO26m (NMS)	kitti_test	3769	MPS	20.4	20.5	154	1.8

Table 4: Per-run latency, memory and wall time (batch 1, after 10 warm-up images). Peak memory is the MPS unified-memory allocator peak, or process RSS for CPU runs.

Run	Exact command (from the run's metadata; also in `runs/COMMANDS.md`)
Faster R-CNN v2, `coco_val`	`python -m hw1.run --model frcnn_v2 --split coco_val --device mps`
Faster R-CNN v2, `kitti_test`	`python -m hw1.run --model frcnn_v2 --split kitti_test --device mps`
Grounding DINO-T, `coco_val1k`	`python -m hw1.run --model gdino_t --split coco_val1k --device cpu`
Grounding DINO-T, `kitti_test`	`python -m hw1.run --model gdino_t --split kitti_test --device cpu`
OWLv2, `coco_val1k`	`python -m hw1.run --model owlv2 --split coco_val1k --device mps`
OWLv2, `kitti_test`	`python -m hw1.run --model owlv2 --split kitti_test --device mps`
RT-DETR R50, `coco_val`	`python -m hw1.run --model rtdetr_r50 --split coco_val --device mps`
RT-DETR R50, `kitti_test`	`python -m hw1.run --model rtdetr_r50 --split kitti_test --device mps`
YOLO11m, `coco_val`	`python -m hw1.run --model yolo11m --split coco_val --device mps`
YOLO11m, `kitti_test`	`python -m hw1.run --model yolo11m --split kitti_test --device mps`
YOLO11m (img1280), `kitti_test`	`python -m hw1.run --model yolo11m --split kitti_test --device mps --tag img1280 --set imgsz=1280`
YOLO26m (NMS-free), `coco_val`	`python -m hw1.run --model yolo26m --split coco_val --device mps`
YOLO26m (NMS), `coco_val`	`python -m hw1.run --model yolo26m --split coco_val --device mps --tag nms --set end2end=false`
YOLO26m (NMS-free), `kitti_test`	`python -m hw1.run --model yolo26m --split kitti_test --device mps`
YOLO26m (NMS), `kitti_test`	`python -m hw1.run --model yolo26m --split kitti_test --device mps --tag nms --set end2end=false`

Table 5: Commands for every main run. Each also records checkpoint, input rule, thresholds, library versions and a source hash in `runs/preds/<run>.meta.json`.